

Sri Lanka Institute of Information Technology



<Exercise 1>

<IT25102533>

<I.K Divesh>

Artificial Intelligence and Machine Learning

IT2011

B.Sc. (Hons) in Information Technology

1.

- My usual favorite, Gemini (specifically Gemini 3.7 Flash), is highly ranked in the top 10 and is noted as a top price-performance model.

2.,3. & 4.

1. Model: Qwen 3.7 <https://chat.qwen.ai/>

- Prompt: Can you explain the concept of method overloading vs method overriding in object oriented programming.
- Response:

💡 Thinking completed >

Method overloading and method overriding are two fundamental concepts in Object-Oriented Programming (OOP) that both deal with **polymorphism** (the ability of a method to take on different forms). However, they are used in completely different scenarios and work in different ways.

Here is a breakdown of each concept, how they differ, and examples to illustrate them.

1. Method Overloading (Compile-Time Polymorphism)

Method Overloading occurs when a class has **multiple methods with the same name, but different parameters** (different number of parameters, different types, or a different order of types).

- **Where it happens:** Within the *same* class.
- **Purpose:** To increase the readability of the program. It allows you to use the same method name to perform similar actions on different types or amounts of data.
- **When it is resolved:** At **compile-time**. The compiler looks at the method name and the arguments you pass to decide which specific version of the method to execute.

Example (Java/C# style):

Imagine a `Calculator` class. You want to add numbers, but sometimes you want to add two numbers, sometimes three, and sometimes decimals.

```
java
1 class Calculator {
2     // Method 1: Adds two integers
3     int add(int a, int b) {
4         return a + b;
5     }
6
7     // Method 2: Overloaded! Same name, different number of parameters
8     int add(int a, int b, int c) {
9         return a + b + c;
10    }
11
12    // Method 3: Overloaded! Same name, different parameter types
13    double add(double a, double b) {
14        return a + b;
15    }
16 }
```

If you call `add(5, 10)`, the compiler knows to use Method 1. If you call `add(5.5, 10.2)`, it uses Method 3.

Analogy: Think of the word "Book". It can be a noun (a thing you read) or a verb (to reserve a flight). The word is the same, but its behavior changes based on the context (parameters) around it.

2. Method Overriding (Run-Time Polymorphism)

Method Overriding occurs when a subclass (child class) provides a specific implementation for a method that is already defined in its superclass (parent class).

- **Where it happens:** Between *two classes* that have an inheritance relationship (Parent and Child).
- **Purpose:** To allow a subclass to customize, replace, or extend the behavior of an inherited method.
This is the core of **Run-Time Polymorphism**.
- **When it is resolved:** At **run-time**. The JVM (or runtime environment) looks at the *actual object* being referenced, not the reference type, to decide which method to call.
- **Rule:** The method name and the parameter list **must be exactly the same** as the parent class.

Example:
Imagine a generic `Animal` class, and specific subclasses like `Dog` and `Cat`.

```
java
1 class Animal {
2     void makeSound() {
3         System.out.println("Some generic animal sound");
4     }
5 }
6
7 class Dog extends Animal {
8     // Overriding the parent method
9     @Override
10    void makeSound() {
11        System.out.println("Bark! Bark!");
12    }
13 }
14
15 class Cat extends Animal {
16     // Overriding the parent method
17     @Override
18    void makeSound() {
19        System.out.println("Meow!");
20    }
21 }
```

How Run-Time Polymorphism works here:

```
java
1 Animal myPet = new Dog();
2 myPet.makeSound(); // Outputs: "Bark! Bark!"
```

Even though the variable `myPet` is of type `Animal`, at *run-time*, the program looks at the actual object in memory (`new Dog()`) and executes the `Dog` class's overridden version of the method.

Analogy: Think of a corporate rule: "All employees must *Work*." (Parent class). A *Software Engineer* overrides "Work" to mean "Write Code". A *Chef* overrides "Work" to mean "Cook Food". The action name is the same, but the specific execution depends on who is doing it.

Summary Comparison Table

Feature	Method Overloading	Method Overriding
Location	Happens within the same class .	Happens between two classes (Parent & Child).
Method Name	Must be the same .	Must be the same .
Parameters	Must be different (type, number, or order).	Must be exactly the same .
Return Type	Can be different (but params <i>must</i> differ).	Must be the same (or a subtype/covariant).
Binding Time	Compile-time (Static binding).	Run-time (Dynamic binding).
Primary Purpose	Code readability and convenience.	Achieving polymorphism and customizing inherited behavior.
Inheritance	Not required.	Required (needs a Parent-Child relationship).

A Quick Trick to Remember

- **Overloading** = You are **loading** the same method name with **multiple** different parameter sets.
- **Overriding** = You are **rewriting** (riding over) the parent's method to give it a **new** behavior for the child class.

2. Model: Longcat <https://longcat.chat/>

- Prompt: How do I write a simple for loop in Java to print out every item in an ArrayList? Just the code please
- Response:

Here are two simple ways to write a for loop in Java to print every item in an ArrayList:

Enhanced for loop (recommended):

▼ java

Copy

```
for (String item : list) {  
    System.out.println(item);  
}
```

Traditional for loop:

▼ java

Copy

```
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i));  
}
```

